Advanced Topics in Hardware Accelerators

HW1 – Matrix Multilication

Student 1: Bashar Abu Leil | 212192181 | bashar.ab@campus.technion.ac.il.

Student 2: Rami Abu-Mukh | 324276765 |ramiab@campus.technion.ac.il.

Student 3: Tom Ostfeld | 212363329 | tom.ostfeld@campus.technion.ac.il

Matrix Multiplication

Introduction:

Matrix multiplication is a cornerstone operation in numerous scientific, engineering, and machine learning applications. As the scale and complexity of modern workloads grow particularly with the rise of large language models (LLMs) and artificial intelligence (AI), efficient matrix computation becomes critical. These workloads often involve massive numbers of dense matrix multiplications, making this operation a common performance bottleneck.

To meet these demands, hardware accelerators such as NVIDIA GPUs are widely adopted, offering massive parallelism through the CUDA programming model. In this context, optimizing matrix multiplication on GPUs is essential to fully exploit their computational power.

In this assignment, we developed a highly optimized CUDA kernel tailored for batched matrix multiplication of 32×32 matrices. Our objective was to accelerate the multiplication of 10,000 matrix pairs using an NVIDIA RTX 2080 Ti GPU, while preserving float32 precision and adhering to the constraints of a fixed benchmarking framework. The ultimate goal was to approach or exceed the performance of PyTorch's highly tuned `bmm` (batched matrix multiplication) implementation.

Our optimization strategy centered on maximizing memory throughput and compute utilization. We leveraged GPU architectural features such as shared memory for fast data access and thread-level parallelism to achieve high occupancy. Specifically, we implemented a quadruple buffer tiled kernel that overlaps global memory loads with arithmetic operations, effectively hiding memory latency. By assigning one CUDA block per matrix multiplication and using a 32×32 thread configuration (one thread per output element), we ensured full coverage of the computation with minimal synchronization overhead.

This approach yielded a mean runtime of approximately **0.07** milliseconds per matrix multiplication, representing a significant improvement over naive CUDA kernels and offering better performance relative to established q like PyTorch. The results demonstrate the potential of low-level GPU programming to extract fine-grained performance gains in matrix-heavy workloads.

Implementation and Methods:

Our implementation performs batched matrix multiplication of 32×32 floating-point matrices using CUDA. The method is designed to optimize memory access and computational efficiency on the GPU by leveraging shared memory and parallel processing.

Methodology and Algorithm

Each matrix multiplication is computed by a single CUDA thread block consisting of 1024 threads, arranged in a 32×32 2D layout. Each thread is responsible for computing a single element in the resulting 32×32 matrix. To improve throughput, each block is assigned a group of four matrix multiplications (denoted as a "cycle") rather than processing one at a time. This batching strategy allows shared memory to be reused across multiple multiplications, improving memory locality and reducing latency.

At the start of the kernel execution, each thread loads one element from each of the four matrices in the cycle into shared memory arrays. This prefetching step reduces global memory traffic, as shared memory access is significantly faster. After synchronization, each thread performs the standard dot-product to compute its assigned output element:

$$\mathrm{C_{i,j}} = \sum_{k=0}^{31} A_{i,k} \cdot B_{k,j}$$

This is repeated for each of the four matrix pairs in the cycle, and results are written back to global memory.

The total number of matrix multiplications is divided by the cycle size (CYCLE_SIZE = 4) to determine the grid size for kernel launch. This batching strategy balances shared memory usage and hardware occupancy.

We chose to use prefetching of 4 pairs of matrixes after checking the NVIDIA GeForce RTX 2080 Ti shared memory size and calculating the size of a single matrix.

This way we were able optimize the fast shared memory capacity.

Motivation

From a theoretical perspective, matrix multiplication is a compute-bound operation with $O(n^3)$ complexity for naive algorithms. Optimizing data locality—particularly reusing matrix elements stored in shared memory—is key to achieving high performance on GPU architectures. The use of shared memory allows for low-latency reuse of matrix tiles, reducing the number of global memory accesses.

Empirically, batching four matrix multiplications per thread block improves GPU utilization and throughput. It enables better instruction-level parallelism and ensures higher streaming multiprocessor (SM) occupancy. This design reduces synchronization overhead and maximizes data reuse, leading to faster execution times in practice.

Results and Discussion:

The overall mean computation time per matrix multiplication achieved by our implementation is **0.071 ms**, as shown below.

```
overall mean computation time:  0.07116768062114717   ms
overall mean L1 error:  0.0
overall mean L2 error:  0.0
```

Initially, we implemented a naive approach where a separate kernel was launched for each matrix multiplication. This resulted in an average computation time of approximately **1.5 ms** per matrix multiplication.

We then compared our optimized implementation with PyTorch's built-in batched matrix multiplication function, which achieved a mean computation time of **0.085 ms** per matrix multiplication.

The observed performance gain stems from the use of fixed-size batches and the ability to tailor the implementation to the specific GPU architecture (NVIDIA RTX 2080 Ti), allowing for efficient memory usage and high parallelism.